Timothy Fitch

CSPB 3112

Final Project Report

Github Link: https://github.com/timothyfitchcupb/timothyfitchcupb.github.io

Project Website: https://timothyfitchcupb.github.io/

Healthcare Worker Advocacy CRM

Introduction

My goal for this project was to design and implement a backend system for managing nonprofit advocacy data. The goal was to strengthen my skills in database design, backend programming, and working with structured data in a way that was practical and relevant.

I specifically aimed to design a system that tracks members, campaigns, and interactions, and supports a full data pipeline. From inputting data, to storing it in a relational database, and retrieving and filtering it through an API, generating summary reports, and exporting those results to CSV.

Background

This project is based on real world advocacy workflows. From my experience with healthcare and union related advocacy efforts, I have seen and experienced firsthand that outreach and organizing data is often managed in spreadsheets or manual systems. These approaches are difficult to maintain, especially when data is extra large or more detailed types of reporting are needed. I have personally had experiences with scattered and disconnected systems and they waste an incredible amount of time and effort to deal with.

To offer improved organization and offer manageable scalability is the purpose of this project. By using a relational database and API the system is able to provide a realistic and flexible solution to storing data.

From a professional development standpoint this project has helped me apply database and backend development concepts in a way that seems practical and relevant to lived experience. It also greatly improved my understanding of how real world systems are designed and implemented

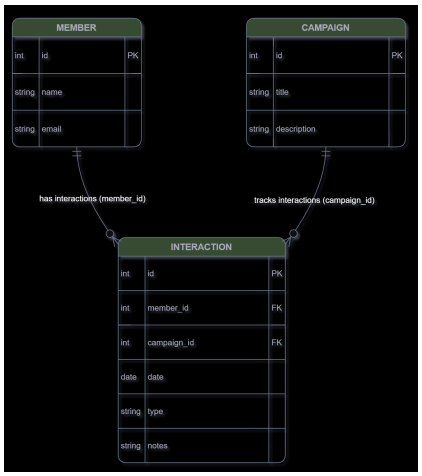
Methodology, Materials and Methods

The development of this project was guided by a combination of official relevant documentation, practical tutorials, and lots of iterative hands on implementation.

The primary framework I used was FastAPI, with its official documentation serving as a main key reference for building the API endpoints, handling requests and responses, and structuring the backend. SQLite also was used as the database due to its simplicity and how easy it is to integrate, and SQLAlchemy was used to define the models and manage the database interactions.

In addition to all of the official documentation, I referenced other practical resources such as FastAPI tutorials, AQLAlchemy database integration guides, and REST API design best practices. I used these resources to help inform how to structure endpoints, manage data relationships, and design a clean as well as consistent API.

Many of the general database design concepts, including entity relationships and foreign keys, were reinforced by using ER diagrams and applying them directly in the design of the Member, Campaign, and Interaction entities.



For my incremental development, the approach followed was essentially:

- Build minimal working system
- Expand features step by step
- Test each of the components continuously

Using this approach ensured that each part of the system functioned correctly before moving on to more advanced features that came later like reporting and CSV export.

Results

1. What I Learned

Throughout this project, I believe I have gained a strong understanding of how backend systems can be structured and how different components interact with each other. I learned how to design a relational database, implement API endpoints, and connect application logic so that the data can be stored, retrieved, and updated functionally.

I also developed a better understanding of how relationships between entities function in a real life system, particularly through the use of foreign keys and structured data models. In addition to this, I learned how to implement reporting features and export data in an easy to use and comprehend format. This all connects the backend development to the needs this database would be applied to address in real life.

2. How I know I Learned It

I have demonstrated what I have learned through the successful implementation of this complete backend system. The API fully supports CRUD (Create/Read/Update/Delete) operations for members, campaigns, and interactions, and all of the data is stored and retrieved correctly from the database.

The system also includes working reporting endpoints that aggregate the interaction data by member and by campaign, as well as a CSV export feature that allows the data to be used outside of the confines of this application, which in a setting like healthcare advocacy will be used extensively.

These features show that I not only was able to get my mind around the concepts and structure required here but that I was also able to apply them in a way that shows that I understand their purpose as a functional tool and not just as a coding project.

3. What were your project assessments, and did you reach those goals?

The project at the current moment does successfully meet all the initial goals I set out to achieve. The backend is fully functional, has clear data organization, working API endpoints, and reporting and exporting capabilities.

Putting together a system that supports a fully developed data pipeline with the ability to input data, store it, change it, retrieve it and filter it as well as generating summary reports and export features was a lot to get through, but I am happy with how it was implemented. While the

system is intentionally simple it accurately reflects a real world backend structure and functionality.

Reflection and Conclusion

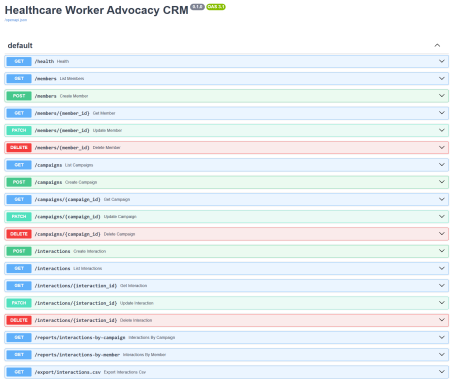
Yes, I believe I met my goals for this project. The most important thing for me was creating and eventually maintaining a structured and incremental development process. By being able to build a simple foundation first and gradually adding features, I was able to avoid any real major prohibitive roadblocks and ensure that each of the components of the system was working correctly before moving on to expansion. For example, I encountered validation errors (HTTP 422) when request data did not match model definitions, which I was able to figure out and resolve by adjusting schema design.

One of the most important things I learned and what I will now take with me in my professional life going forward is just getting into this process faster and more smoothly. It took me a few weeks to combine time management and the restrictions of the course with the ability to successfully develop at a more methodical and steady pace that eventually led to weekly success and then to overall success.

Overall I am really happy with the results of the project. I had a clear vision from the start of what I wanted to create and what I wanted the use case of that creation to be, and I was able to successfully achieve that vision with a working product. I have created an exact system which at one point I was begging for in my career, and that certainly feels like a good place to be.

To expand on this project in the future, I would love to be able to expand the frontend or user interface. The current system clearly has a developed and working backend, but to really make this feel like a fully fleshed creation there needs to be a user interface. There is also the option to expand reporting capabilities with even more advanced queries. Another angle for addition would be integrating authentication and user management as well as implementing cybersecurity tools as this data needs to be secured in any of the real world use cases this product would be used in. One thing I really wanted to include was a full video demo showcasing a full run through of the system with voiceover, I even wrote a test script, but I just did not have the time given the restraints of the class.

Overall, looking towards the future I am excited about what I am able to do, I know I have the skills to conceive a project and structure it visually, break it down into manageable chunks that prioritize time management and time efficiency, do the research and code implementation necessary to achieve the project through many series of iterative building all to create something that is functional.



	A	B	C	D	E	F	G	H	I	J
1	id	member_id	campaign_	date	type	notes				
2	2	1	1	4/20/2026	event	Attended healthcare advocacy town hall				
3	1	1	1	4/21/2026	email	Sent outreach email regarding safe staffing legislation				
4										

References

Official Documentation

FastAPI Documentation
https://fastapi.tiangolo.com/

FastAPI – SQL (Relational) Databases Tutorial
https://fastapi.tiangolo.com/tutorial/sql-databases/

SQLAlchemy Documentation
https://docs.sqlalchemy.org/

SQLite Documentation
https://www.sqlite.org/docs.html

Python Documentation
https://docs.python.org/3/

Tutorials and Learning Resources

How to Build a FastAPI SQLite REST API (CRUD + SQLAlchemy) – YouTube
https://www.youtube.com/watch?v=Z0jbO8WT0Jc

FastAPI + SQLAlchemy Tutorial — Build a REST API – YouTube
https://www.youtube.com/watch?v=xq1Snezb1rs

FastAPI Tutorial – Adding a Database with SQLAlchemy (Corey Schafer) – YouTube
https://www.youtube.com/watch?v=NvOV3ig2tGY

SQLAlchemy Crash Course – Python Database ORM (NeuralNine) – YouTube
https://www.youtube.com/watch?v=529LYDgRTgQ

SQLite Introduction – Beginner Guide to SQL Databases (Caleb Curry) – YouTube
https://www.youtube.com/watch?v=8Xyn8R9eKB8

Articles

FastAPI + SQLite Databases (Step-by-Step Guide) – GeeksforGeeks
https://www.geeksforgeeks.org/python/fastapi-sqlite-databases/

Building a Simple CRUD API with FastAPI and SQLAlchemy – Stackademic
https://blog.stackademic.com/building-a-simple-api-crud-with-python-fastapi-and-sqlalchemy-1ed59de4f2ef